

OWASP Top 10

A man in profile, looking intently at a laptop screen. The background is dark with a grid pattern and the words 'IT SECURITY' in a large, blurred, digital font. The overall mood is serious and focused.

Secure Programming

Michael Lehmann

IT SECURITY

Open Worldwide Application Security Project

Open Worldwide Application Security Project

- «worldwide not-for-profit charitable organization focused on improving the security of software»

Has a lot of initiatives / (sub-)projects:

- [Cheat Sheet Series](#)
- [Dependency Track](#)
- [OWASP SAMM](#)
- [SecurityShepherd](#)
- [Security Knowledge Framework \(SKF\)](#)
- [ZAP](#)
- [ModSecurity Core Rule Set](#)
- [Top10](#)
- ...and many more: [OWASP Projects](#)

OWASP Top 10

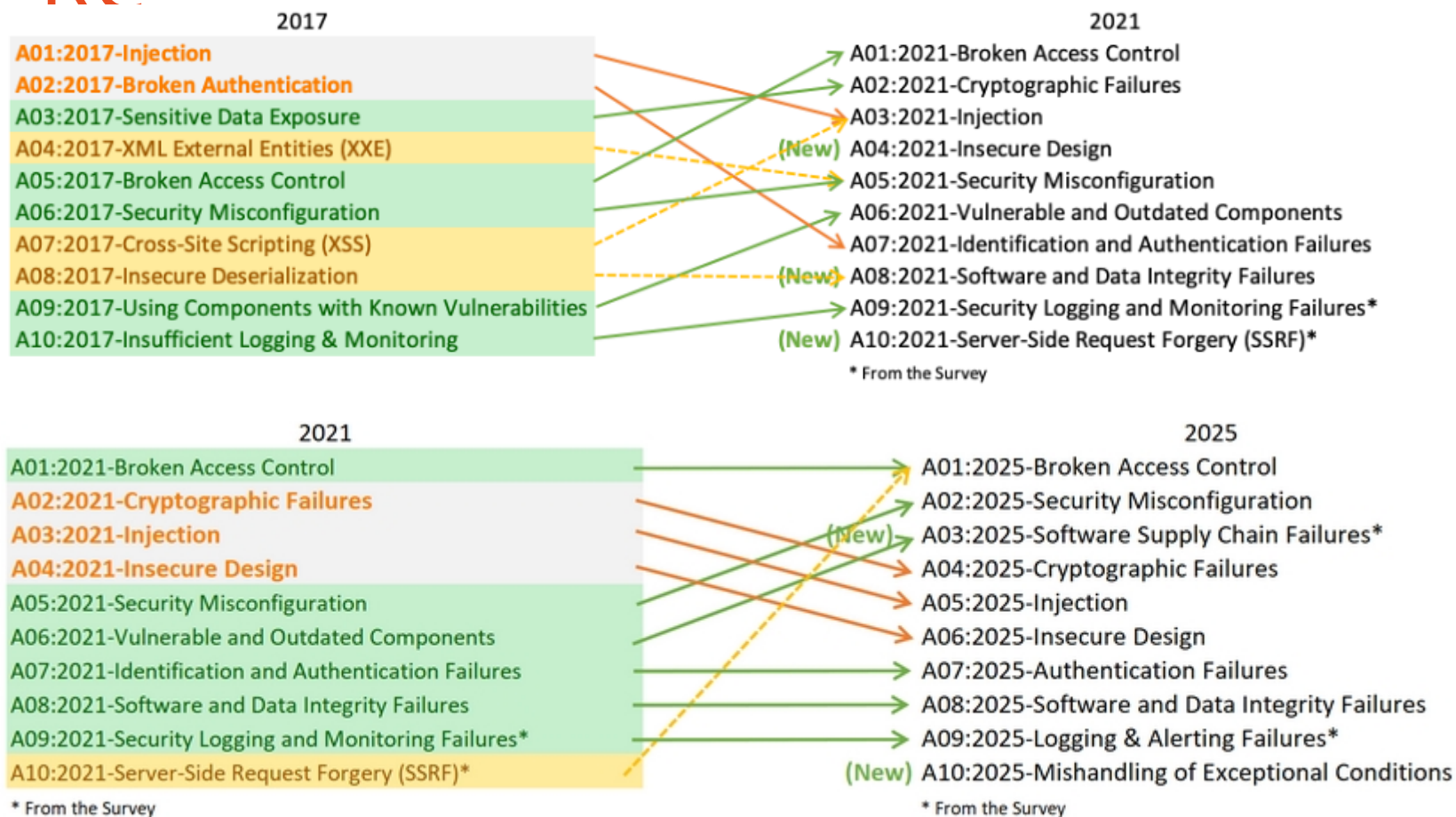
List of most critical security risks

- Technical vulnerabilities
- How to prevent them

Various uses

- Educational purposes
- Checklist for development and threat modelling

OWASP Top 10 2017 vs. 2021 vs. 2025



A02 Cryptographic Failure

Issues

Insufficient encryption in transit

Insufficient encryption at rest

Weak algorithms

Storage of passwords in plaintext, encrypted or unsalted hash

Sensitive data

Passwords

Credit card information

Personal information

Health data

PII

How to avoid Sensitive Data Exposure

Do not store sensitive data

Encrypt sensitive data

Use strong algorithms

[OWASP Cryptographic Storage Cheat Sheet](#)
[NIST Cryptographic Standards and Guidelines](#)

Use secure random
generators

Implement proper key
management

Store passwords with secure
salted hashes

Set HSTS header

A05 Security Misconfiguration

Issues

- Missing or insufficient hardening
- Unnecessary features
- Default accounts
- Detailed error messages
- ...

Countermeasures

- Repeatable hardening and deployment
- Minimal platform
- Review configurations
- Segmented application architecture
- ...

XML External Entities are part of A05



- XML Entities are key/value pairs
 - e.g. < → <



- DTD contain declarations
 - Can be internal or external



- Custom entity definitions possible
 - <!DOCTYPE foo [<!ENTITY myentity "value" >]>



- Can make http requests
 - <!DOCTYPE foo [<!ENTITY ext SYSTEM "http://something.com" >]>

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///etc/shadow" >]><foo>&xxe;</foo>
```


XXE can lead to various issues

Issues

- Data leak & exfiltration
- DoS
- RCE
- SSRF

Countermeasures

- Using other formats like JSON
- Disable processing of XXE, DTD
- Update components
- Validate XML & XSL

A06 Vulnerable and Outdated Components

Issues

- Outdated components
- Unsupported components
- Insufficient patching
- Insufficient testing

Countermeasures

- Remove unnecessary things
- Update regularly and risk-based
- Continuously monitor components
- Obtain components from official sources

A08 Software and Data Integrity

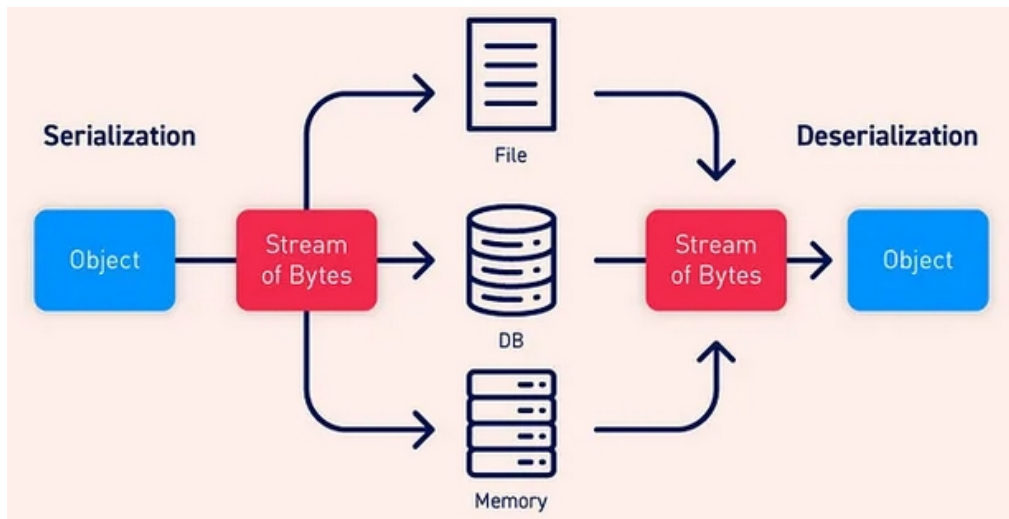
Issues

- Unverified software packages
- Ingestion of untrusted data
- Insecure CI/CD configuration

Countermeasures

- Verify signatures of data and software
- Use trusted repositories
- Establish review process
- Check for known vulnerabilities
- Ensure secure CI/CD

Serialisation is often used for persistence



Countermeasures

- Avoid deserialization
- No untrusted sources
- Integrity checks
- Type constraints
- Isolate
- Log exceptions and failures
- Restrict and monitor network

A09 Security Logging and Monitoring Failures

Issues

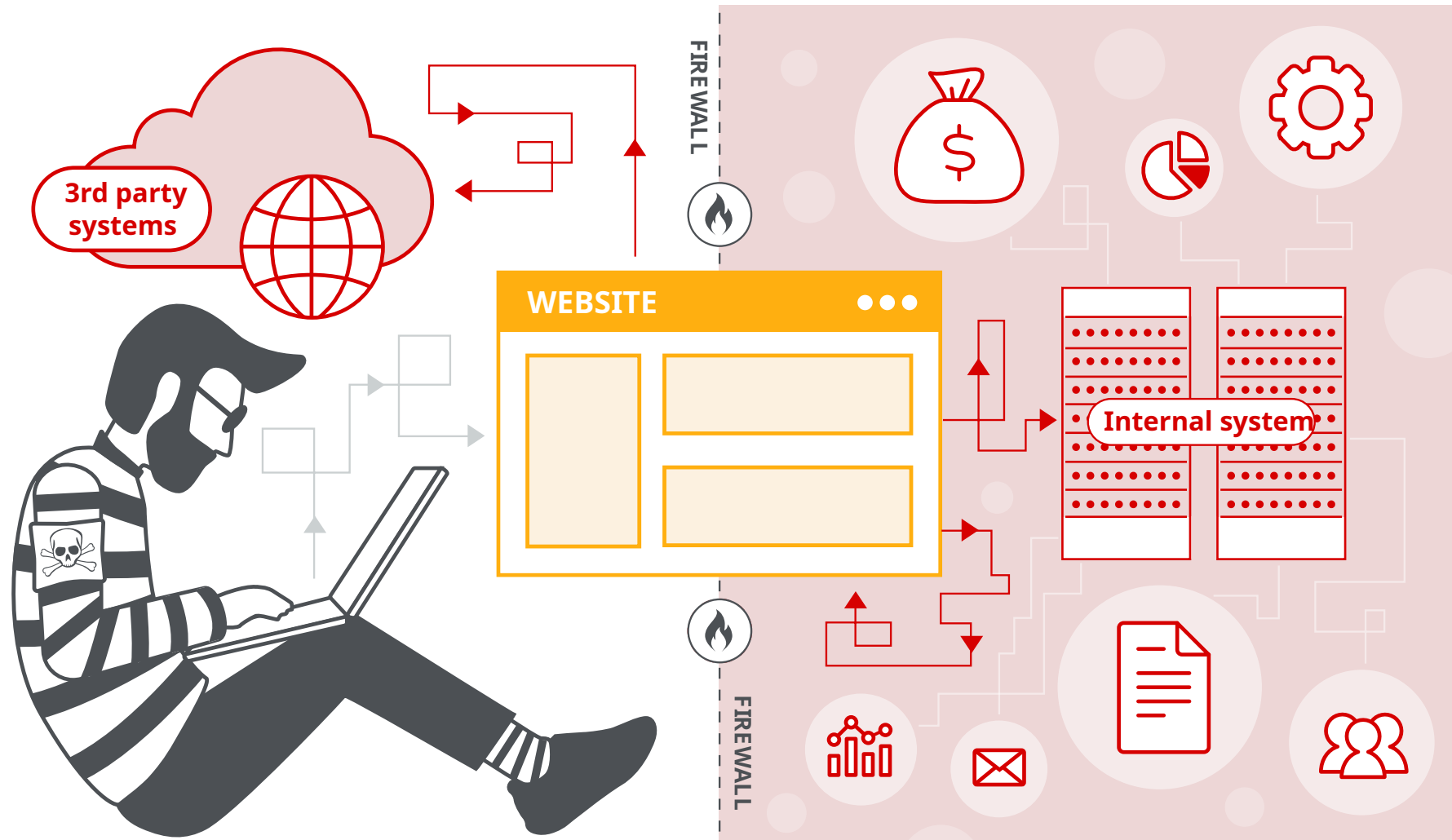
- Insufficient logs
- No monitoring & alerting
- Local storage only
- Missing integrity checks

Countermeasures

- Audit trail
- Integrity and proper encoding
- Centralised logs and normalised format
- Effective monitoring and alerting
- Incident response and recovery plan

Application layer intrusion detection and automated response: [OWASP AppSensor Project](#)

A10 Server Side Request Forgery



A10 Server Side Request Forgery

Countermeasures

- No user-supplied input in URLs
- Whitelist-based input validation
- Network segregation

OWASP API Security Project

Security risks focusing on modern API based applications

OWASP API Security

API1:2023	Broken Object Level Authorization
API2:2023	Broken Authentication
API3:2023	Broken Object Property Level Authorisation
API4:2023	Unrestricted Resource Consumption
API5:2023	Broken Function Level Authorization
API6:2023	Unrestricted Access to Sensitive Business Flows
API7:2023	Server Side Request Forgery
API8:2023	Security Misconfiguration
API9:2023	Improper Inventory Management
API10:2023	Unsafe Consumption of APIs

AP3 Broken Object Property Level Authorisation

Issues

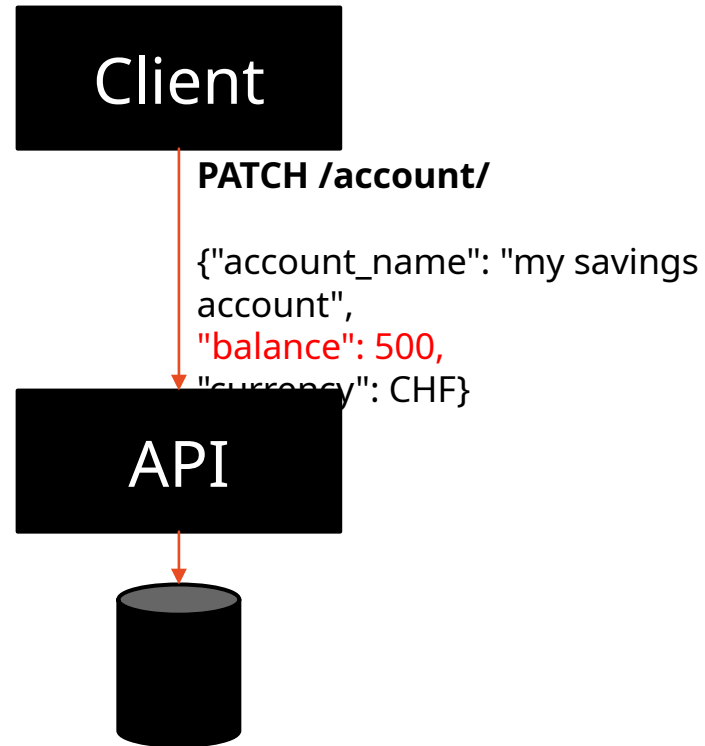
- Sensitive properties returned
- Sensitive properties modifiable

Countermeasures

- Verify access rights
- Return specific values only
- Avoid direct binding
- Validate responses
- Minimise data returned

Mass Assignment

- ▶ API endpoint may allow to modify data sent from the client (by design)
 - ▶ E.g. modify bank account settings
- ▶ Mass assignment exploitation allows to modify data that would not be permitted
 - ▶ E.g. account balance
- ▶ Exploiting missing protection on specific data model access (fields or objects)



Mass Assignment Example

- ▶ A form for editing user information

```
<form>
  <input name="userid" type="text">
  <input name="password" type="text">
  <input name="email" type="text">
  <input type="submit">
</form>
```

- ▶ Corresponding object

```
public class User {
  private String userid;
  private String password;
  private String email;
  private boolean isAdmin;

  //Getters & Setters
}
```

Mass Assignment Example

▶ Binding

```
@RequestMapping(value = "/addUser", method = RequestMethod.POST)
public String submit(User user) {
    userService.add(user);
    return "successPage";
}
```

▶ Normal request

```
POST /addUser
...
userid=bobbytables&password=hashedpass&email=bobby@tables.com
```

▶ Exploit

```
POST /addUser
...
userid=bobbytables&password=hashedpass&email=bobby@tables.com&isAdmin=true
```

API4 Unrestricted Resource

Issues

- DoS
- Unnecessary costs

Countermeasures

- Define limits
- Enforce maximum data size
- Scale where needed
- Validate input
- Spending limits/alerts

API6 Unrestricted Access to Sensitive Business Flows

Issues

- Loss of business
- Business specific

Countermeasures

- Fingerprinting: deny for unexpected devices
- Human detection
- Non-human detection
- Block known proxies
- Etc.

API9 Improper Inventory Management

Issues

- Missing or outdated inventory
- Missing or outdated documentation
- Unfinished or deprecated APIs exposed
- Unclear data flows

Countermeasures

- Inventory API hosts and integrated services
- Document everything
- Automate generation of documentation
- Use external protection measures
- Include old versions in patches

API10 Unsafe Consumption of APIs

Issues

- Sensitive data exposure
- Injection
- Open redirect
- DoS

Countermeasures

- Security assessment
- Ensure encryption
- Perform input validation
- Whitelist redirects

Exercise

Identify the problem

1. What's wrong here?
2. How can you fix the issue?

python

 Copy code

```
def login(username, password):  
    user = database.getUserByUsername(username)  
  
    if user is None:  
        return "User not found"  
  
    if user.password == password:  
        return "Login successful"  
    else:  
        return "Incorrect password"
```

Identify the problem

1. What's wrong here?
2. How can you fix the issue?

python

 Copy code

```
from flask import Flask, request, render_template
import requests

app = Flask(__name__)

@app.route('/fetch', methods=['GET'])
def fetch_url():
    url = request.args.get('url')
    response = requests.get(url)
    return render_template('result.html', content=response.text)

if __name__ == '__main__':
    app.run()
```

Exercises

- ▶ <https://overthewire.org/wargames/natas/>

